3.8、图像金字塔

[OpenCV：图像金字塔]
https://docs.opencv.org/4.x/dc/dff/tutorial_py_pyramids.html

图像金字塔是 OpenCV 中非常重要的多分辨率技术，常用于目标检测（不知道物体大小）、图像融合（无缝拼接）、图像压缩等场景。

1. 什么是 Image Pyramids?

把同一张图片做成不同分辨率的版本，从高分辨率（底部）到低分辨率（顶部）堆叠起来，看起来像一座金字塔。主要分成两种：

- Gaussian Pyramid（高斯金字塔）：用于降低/升高分辨率。
- Laplacian Pyramid（拉普拉斯金字塔）：用于捕捉图像的细节（像边缘图），常用于图像融合。

2. Gaussian Pyramid（高斯金字塔）

向下（缩小）：

- 使用 cv2.pyrDown()：先做高斯模糊，再把图像的长宽各缩小一半（面积变成原来的 1/4）。
- 每缩小一次称为一个 octave（八度）。
  向上（放大）：
- 使用 cv2.pyrUp()：先把图像放大一倍（插入像素），再做高斯模糊。
- 注意：**放大后再缩小，无法完全恢复原图，因为信息在缩小过程中丢失了**。

```
import cv2 as cv
import numpy as np

img = cv.imread('messi5.jpg')   # 替换成你的图片路径
assert img is not None, "图片读取失败"

# 生成高斯金字塔（向下6次）
gp = [img]          # 第0层是原始图像
G = img.copy()
for i in range(6):
    G = cv.pyrDown(G)
    gp.append(G)
    cv.imshow(f'Gaussian Level {i+1}', G)
```

```
cv.waitKey(0)
cv.destroyAllWindows()
```

你可以尝试用 cv.pyrUp() 把某层再放大回去对比差异。

3. Laplacian Pyramid（拉普拉斯金字塔）

拉普拉斯金字塔 没有直接函数，需要从高斯金字塔构建。
构建方法： 每一层 = 当前高斯层 - (上一层高斯放大后的版本)
公式大致是：

$$L_i = G_i - \mathrm{pyrUp}(G_{i+1})$$

拉普拉斯图像大部分像素接近 0，看起来像边缘/细节图，非常适合压缩和融合。代码示例（构建 Laplacian 金字塔）假设你已经有 gpA（高斯金字塔列表，第0层最高分辨率，第5层最低）：

```
# 从最粗糙层开始构建 Laplacian
lp = [gpA[5]]                    # 最顶层直接用高斯层
for i in range(5, 0, -1):
    GE = cv.pyrUp(gpA[i])        # 把上一层放大
    L = cv.subtract(gpA[i-1], GE)  # 减去放大后的版本
    lp.append(L)
```

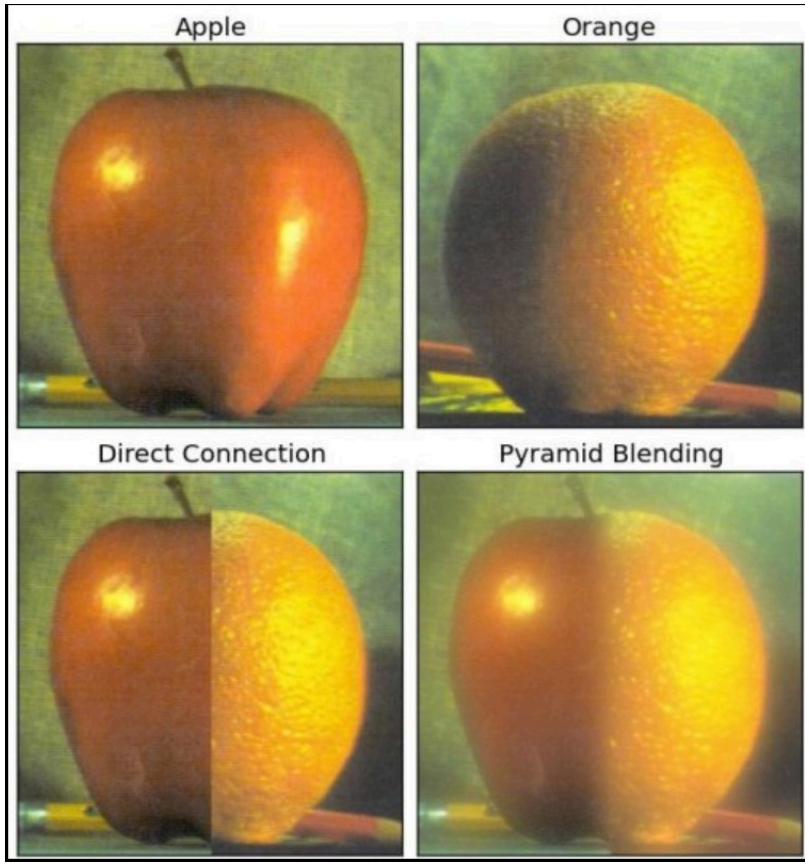
4. 经典应用：Image Blending（图像融合）—— “Orapple”（苹果 + 橙子）

直接把两张图片左右拼起来会有明显接缝。使用拉普拉斯金字塔可以实现无缝融合：在不同频率（分辨率）层次分别融合，最后重建图像。完整融合代码（官方示例，6层金字塔）

```
import cv2 as cv
import numpy as np

# 读取两张同样大小的图片（苹果和橙子）
A = cv.imread('apple.jpg')
B = cv.imread('orange.jpg')

assert A is not None and B is not None, "图片读取失败"

# 1. 构建两张图的高斯金字塔（6层）
def gaussian_pyramid(img, levels=6):
    G = img.copy()
    gp = [G]
    for i in range(levels):
```

```python
        G = cv.pyrDown(G)
        gp.append(G)
    return gp

gpA = gaussian_pyramid(A)
gpB = gaussian_pyramid(B)

# 2. 从高斯金字塔构建拉普拉斯金字塔
def laplacian_pyramid(gp):
    lp = [gp[-1]]                       # 最顶层
    for i in range(len(gp)-2, -1, -1):
        GE = cv.pyrUp(gp[i+1])
        L = cv.subtract(gp[i], GE)    # 细节层
        lp.append(L)
    return lp

lpA = laplacian_pyramid(gpA)
lpB = laplacian_pyramid(gpB)

# 3. 在每一层拉普拉斯中融合（左半苹果 + 右半橙子）
LS = []
for la, lb in zip(lpA, lpB):
    rows, cols, dpt = la.shape
    ls = np.hstack((la[:, 0:cols//2], lb[:, cols//2:]))  # 左右拼接
    LS.append(ls)

# 4. 重建最终图像
ls_ = LS[0]
for i in range(1, len(LS)):
    ls_ = cv.pyrUp(ls_)
    ls_ = cv.add(ls_, LS[i])

# 对比直接拼接的效果
real = np.hstack((A[:, :A.shape[1]//2], B[:, B.shape[1]//2:]))

cv.imwrite('Pyramid_blending.jpg', ls_)
cv.imwrite('Direct_blending.jpg', real)

cv.imshow('Pyramid Blending', ls_)
cv.imshow('Direct Blending', real)
cv.waitKey(0)
```

效果对比：金字塔融合后接缝几乎看不见，而直接拼接会有明显边界。



5.金字塔中append()

5.1. list.append() 的基本作用

append() 永远是在列表的末尾（最后面）添加一个新元素。例如：

```
ls = []
ls.append("A")   # ls = ["A"]
ls.append("B")   # ls = ["A", "B"]
ls.append("C")   # ls = ["A", "B", "C"]
```

所以列表的索引0 总是最早加入的元素，最后一个元素（ls[-1]）总是最新加入的。

5.2. 高斯金字塔（Gaussian Pyramid）中的 append

```
gp = [img]          # gp[0] = 原始图像（最高分辨率，最底层）
G = img.copy()
for i in range(6):
    G = cv.pyrDown(G)        # 每次缩小一半 → 分辨率更低
    gp.append(G)             # ← 这里 append
```

结果列表顺序：

- gp[0] → 最高分辨率（金字塔的最底层，最清晰）
- gp[1] → 缩小1次
- gp[2] → 缩小2次
- ...
- gp[6]（最后一个）→ 最低分辨率（金字塔的最顶层，最模糊）
  结论：每次 append 加入的都是更低一层（分辨率更低、更接近金字塔顶部）。

5.3. 拉普拉斯金字塔（Laplacian Pyramid）中的 append

```
lp = [gpA[5]]                    # 先把最小的（最模糊的那一层）放进去 → lp[0]
for i in range(5, 0, -1):        # 从粗到细的顺序
    GE = cv.pyrUp(gpA[i])
    L = cv.subtract(gpA[i-1], GE)   # 计算这一层的细节
    lp.append(L)                    # ← 这里 append
```

结果列表顺序（以6层为例）：

- lp[0] = 最粗糙的层（最低分辨率，金字塔最顶层）
- lp[1] = 比上面细一点的细节层
- lp[2] = 更细的细节层
- ...
- lp[5]（最后一个）= 最精细的细节层（最高分辨率，金字塔最底层）
  结论：在构建拉普拉斯金字塔时，append 先加入高层（最顶、最粗糙），后面 append 加入的是越来越低的层（越来越精细）。

5.4. 融合后的 LS 列表

也是同样的顺序

```
LS = []
for la, lb in zip(lpA, lpB):
    ... 融合左右半边 ...
    LS.append(ls)        # ← 同样是 append
```

所以 LS[0] 是最顶层（最小、最粗糙的融合层）
LS[5] 是最底层（最大、最精细的细节融合层）5. 重建代码为什么从 LS[0] 开始?

```
ls_ = LS[0]                          # 从金字塔最顶层（最小图像）开始
for i in range(1, len(LS)):          # i=1,2,3,...,5
    ls_ = cv.pyrUp(ls_)              # 放大一倍
    ls_ = cv.add(ls_, LS[i])         # 加上当前这一层的细节
```

因为列表的顺序是 从顶到底（从粗到细），所以重建必须：

- 从最顶（LS[0]）开始
- 每次放大后，加上列表中下一个（更精细）的细节层
  这就是为什么用 range(1, len(LS)) 而不是反过来。